Python Activity-Based Learning:

Program 1 : University Scholarship Allocation

Problem Statement:

Design a Python program to evaluate scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and family income criteria. Rank eligible students and resolve ties using predefined rules.

Objectives:

- Check aggregate marks and minimum subject marks.

- Verify attendance and family income.

- Identify eligible students.

- Rank eligible students and resolve ties.

Assumptions:

- Aggregate marks ≥ 75%.

- Minimum marks in each subject ≥ 60%.

-Attendance ≥ 75%.

- Annual family income ≤ ₹2,00,000.

- Higher aggregate marks get a higher rank; attendance breaks ties.

Algorithm:

- Start.

-Input the number of applicants.

- Input name, subject marks, attendance, and income.

- Calculate aggregate percentage.

- Check all eligibility conditions.

- Store eligible applicants.

- Rank them by aggregate marks and attendance.

- Display the ranked list.

- Stop.

Flowchart:

Start → Input applicant details → Calculate aggregate percentage → Check eligibility conditions → Store eligible students → Rank by marks and attendance → Display ranked list → Stop

Pseudocode:

```
BEGIN
  INPUT number of applicants
  CREATE empty eligible list

  FOR each applicant
    INPUT name
    INPUT subject marks
    INPUT attendance
    INPUT annual family income

    CALCULATE aggregate percentage

    IF aggregate >= 75 AND
      all subject marks >= 60 AND
      attendance >= 75 AND
      income <= 200000 THEN

       ADD applicant to eligible list
       DISPLAY "Eligible"

    ELSE
      DISPLAY "Not Eligible"
```

```
    END IF
  END FOR

  SORT eligible students by aggregate marks
  IF marks are equal, rank by attendance

  DISPLAY scholarship ranking
END
```

Source code:

```
n = int(input("Enter number of applicants: "))
eligible = []

for i in range(n):
  name = input("Enter student name: ")
  m1 = int(input("Enter subject 1 marks: "))
  m2 = int(input("Enter subject 2 marks: "))
  m3 = int(input("Enter subject 3 marks: "))
  attendance = float(input("Enter attendance: "))
  income = float(input("Enter annual family income: "))

  aggregate = (m1 + m2 + m3) / 3

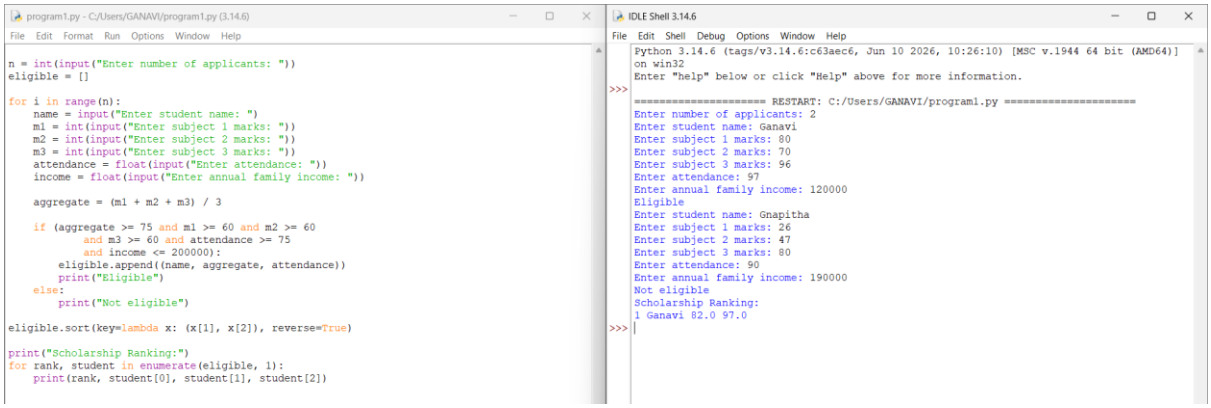
  if (aggregate >= 75 and m1 >= 60 and m2 >= 60
      and m3 >= 60 and attendance >= 75
      and income <= 200000):
    eligible.append((name, aggregate, attendance))
    print("Eligible")
```

```
    else:
        print("Not eligible")


eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)


print("Scholarship Ranking:")
for rank, student in enumerate(eligible, 1):
    print(rank, student[0], student[1], student[2])
```

output:



Testcases:

Test Case	Aggregate Marks	Minimum Subject Marks	Attendance	Annual Family Income	Expected Output
1	85%	80	90%	₹1,50,000	Eligible
2	55%	50	80%	₹1,00,000	Not Eligible
3	80%	65	70%	₹1,80,000	Not Eligible

Conclusion:

The program checks scholarship eligibility using academic marks, attendance, and family income. It ranks eligible applicants based on aggregate marks and attendance.

Program 2:

Multi-Plan Telecommunications Cost Optimization.

problem statement:

Develop a Python program to evaluate several mobile plans based on usage, validity, and budget constraints. Calculate extra charges and identify the most economical eligible plan.

Objectives:

- Compare different mobile plans.
- Calculate extra usage charges.
- Check plan validity and budget constraints.
- Identify the most economical eligible plan.

Assumptions:

- The user's monthly budget is ₹500.
- The user requires at least 28 days of validity.
- Extra data usage costs ₹10 per GB.
- Only plans within the budget and validity requirements are eligible.

Algorithm:

1. Start.
2. Input the number of mobile plans.
3. Input each plan's name, price, validity, included data, and required data usage.
4. Calculate extra data charges.
5. Calculate the total cost of each plan.
6. Check budget and validity conditions.
7. Select the eligible plan with the lowest total cost.

8.Display the best plan and its total cost.

9. Stop.

Flow chart:

Start → Input mobile plan details → Calculate extra usage charges → Calculate total cost → Check budget and validity → Select lowest-cost eligible plan → Display best plan → Stop

Pseudocode:

```
BEGIN
INPUT number of plans
SET budget = 500
SET minimum validity = 28
SET extra charge per GB = 10
SET best plan = NONE

FOR each plan
  INPUT plan name, price, validity
  INPUT included data and required data

  IF required data > included data THEN
    CALCULATE extra data
    CALCULATE extra charges
  ELSE
    SET extra charges = 0
  END IF

  CALCULATE total cost = price + extra charges
```

```
      IF total cost <= budget AND validity >= 28 THEN
        IF best plan is NONE OR
          total cost < lowest cost THEN
           UPDATE best plan
           UPDATE lowest cost
        END IF
      END IF
    END FOR

    IF best plan exists THEN
      DISPLAY best plan and total cost
    ELSE
      DISPLAY "No eligible plan found"
    END IF
END
```

Source code:

```
n = int(input("Enter number of plans: "))

budget = 500
best_plan = None
lowest_cost = 999999

for i in range(n):
    print("\nPlan", i + 1)

    name = input("Enter plan name: ")
```

```
    price = float(input("Enter plan price: "))
    validity = int(input("Enter validity in days: "))
    included_data = float(input("Enter included data in GB: "))
    required_data = float(input("Enter required data in GB: "))

    if required_data > included_data:
        extra_data = required_data - included_data
        extra_charges = extra_data * 10
    else:
        extra_charges = 0

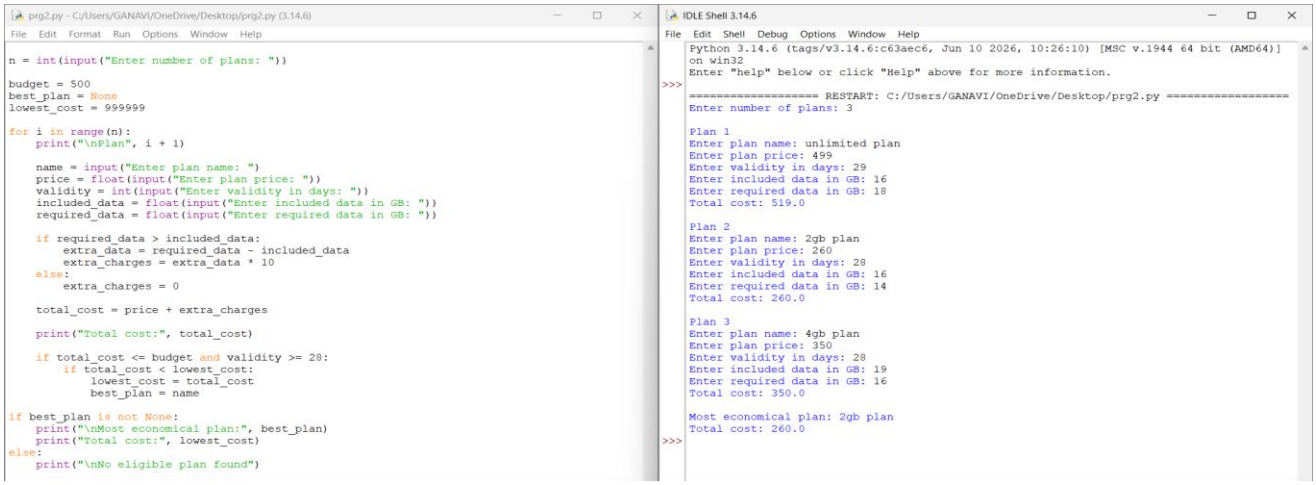
    total_cost = price + extra_charges

    print("Total cost:", total_cost)

    if total_cost <= budget and validity >= 28:
        if total_cost < lowest_cost:
            lowest_cost = total_cost
            best_plan = name

if best_plan is not None:
    print("\nMost economical plan:", best_plan)
    print("Total cost:", lowest_cost)
else:
    print("\nNo eligible plan found")
```

output:



Testcases:

Test case	Plan price	Validity	Data requirement	Expected output
1	199	28days	3 GB required, 2 GB included	Eligible; total ₹209
2	299	56days	4 GB required, 5 GB included	Eligible; total ₹299
3	599	56days	8 GB required, 10 GB included	Not eligible; exceeds budget

Conclusion:

The program compares mobile plans based on usage, extra charges, validity, and budget. It selects the most economical plan that satisfies the given requirements.

Program3

Attendance Recovery Computation

Problem statement:

Develop a Python program to calculate a student's current attendance percentage and find the minimum number of consecutive classes the student must attend to reach the required attendance percentage.

objectives:

- Calculate current attendance percentage.

- Check whether the student meets the attendance requirement.

- Calculate the minimum additional classes required.

-Display the result under different attendance scenarios.

Assumptions:

- Minimum required attendance is 75%.
- Total classes conducted and classes attended are entered by the user.
- The student attends all upcoming classes.
- Attendance is calculate as:
- $\text{Attendance} = \frac{\text{Classes Attended}}{\text{Total Classes}} \times 100$

Algorithm:

1 Start.

2. Input total classes conducted and classes attended.

3. Calculate the current attendance percentage.

4. Check whether attendance is already at least 75%.

5. If not, add one attended class and one total class repeatedly.

6. Continue until attendance reaches 75%.

7. Display the minimum additional classes required.
8. Stop.

Flowchart:

Start → Input total and attended classes → Calculate attendance percentage → Check if attendance ≥ 75% → Calculate additional classes if needed → Display result → Stop

Pseudocode:

```
BEGIN
  INPUT total_classes
  INPUT attended_classes

  CALCULATE attendance percentage
  SET extra_classes = 0

  WHILE attendance percentage < 75
    total_classes = total_classes + 1
    attended_classes = attended_classes + 1
    extra_classes = extra_classes + 1

    CALCULATE attendance percentage
  END WHILE

  DISPLAY current attendance percentage
  DISPLAY extra_classes
END
```

Source code:

```
total = int(input("Enter total classes conducted: "))
attended = int(input("Enter classes attended: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details")
else:
    attendance = (attended / total) * 100

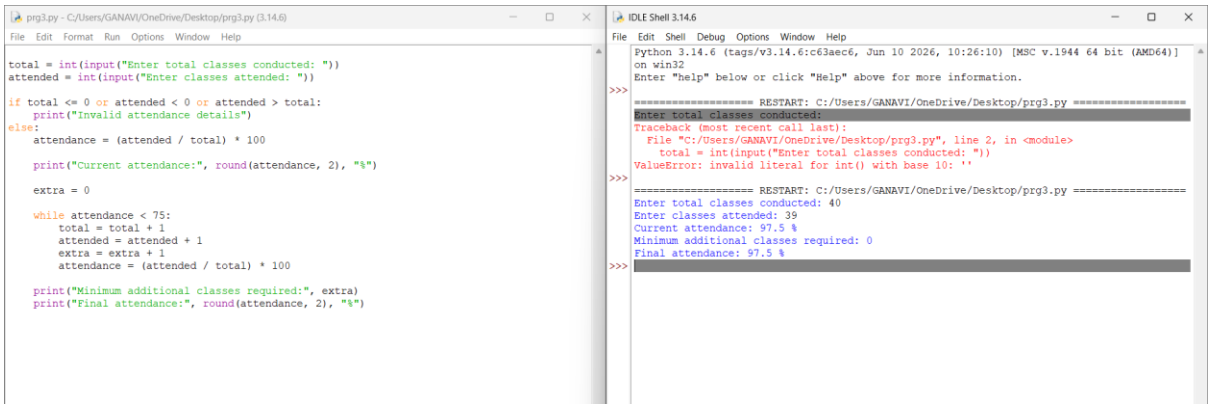
    print("Current attendance:", round(attendance, 2), "%")

    extra = 0

    while attendance < 75:
        total = total + 1
        attended = attended + 1
        extra = extra + 1
        attendance = (attended / total) * 100

    print("Minimum additional classes required:", extra)
    print("Final attendance:", round(attendance, 2), "%")
```

output:



Testcases:

Testcases	Total classes	Attended classes	Additional classes
1	40	28	8
2	50	40	0
3	20	10	3

Conclusion:

The program calculates current attendance and uses a while loop to determine the minimum number of consecutive classes a student must attend to achieve the required 75% attendance.

Program 4

Courier Charge Optimization

Problem statement:

Calculate courier delivery charges based on parcel weight, dimensions, distance zone, urgency, and restrictions. Compare different courier services and select the most suitable eligible service.

Objectives:

- Calculate courier charges.
- Compare different courier services.
- Check parcel restrictions and delivery eligibility.
- Select the lowest-cost eligible courier service.

Assumptions:

- Base charge: ₹50.
- Weight charge: ₹20 per kg.
- Distance charge: ₹10 per zone.
- Express delivery charge: ₹100 extra.

- Maximum parcel weight: 20 kg.
- Select the eligible service with the lowest total cost.

Algorithm:

1. Start.
2. Input parcel details and number of courier services.
3. Calculate the charge for each service.
4. Check weight and restriction conditions.
5. Compare eligible services.
6. Display the cheapest eligible service and its cost.
7. Stop.

Flowchart:

Start → Input parcel and courier details → Calculate delivery charges → Check eligibility → Compare eligible services → Display cheapest service → Stop

Pseudocode:

```
BEGIN
  INPUT parcel_weight, distance_zone
  INPUT delivery_type
  INPUT number_of_services

  SET lowest_cost = large_number
  SET best_service = none

  FOR each service
    CALCULATE total_charge
    CHECK weight and restriction conditions
```

```
    IF service is eligible AND total_charge < lowest_cost
      SET lowest_cost = total_charge
      SET best_service = service_name
    END IF
  END FOR

  DISPLAY best_service and lowest_cost
END
```

Python Source Code:

```
n = int(input("Enter number of courier services: "))

weight = float(input("Enter parcel weight in kg: "))
zones = int(input("Enter distance zones: "))
express = input("Express delivery? (yes/no): ").lower()

lowest_cost = float("inf")
best_service = ""

for i in range(n):
  print("\nService", i + 1)

  name = input("Enter service name: ")
  max_weight = float(input("Enter maximum weight in kg: "))
  restricted = input("Parcel restricted? (yes/no): ").lower()

  if weight > max_weight or restricted == "yes":
    print("Service not eligible")
```

```
        continue

    charge = 50 + (weight * 20) + (zones * 10)

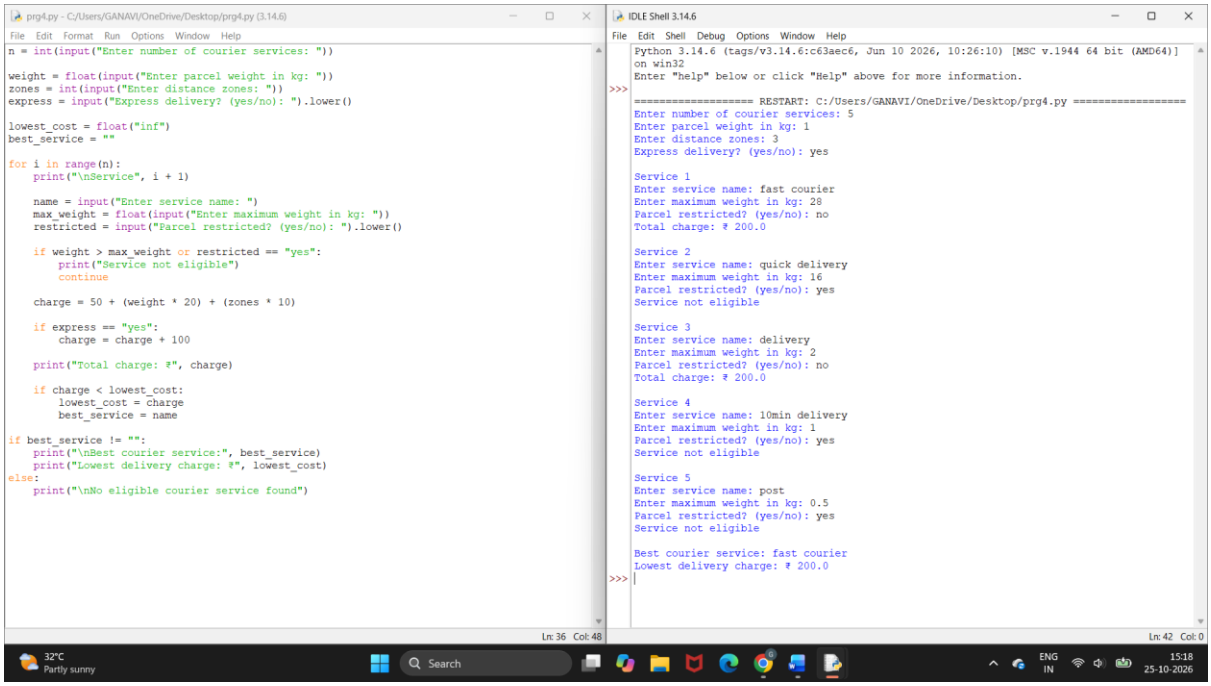
    if express == "yes":
        charge = charge + 100

    print("Total charge: ₹", charge)

    if charge < lowest_cost:
        lowest_cost = charge
        best_service = name

if best_service != "":
    print("\nBest courier service:", best_service)
    print("Lowest delivery charge: ₹", lowest_cost)
else:
    print("\nNo eligible courier service found")
```

output:



Testcases:

Testcases	Input	Expected output
1	Weight 5 kg, 3 zones	Charge ₹180
2	Weight exceeds service limit	Service not eligible
3	Parcel is restricted	Service not eligible

Conclusion:

This Python program calculates courier charges, checks service eligibility, and selects the lowest-cost eligible courier service using conditional statements and loops

Program 5

Repeated Transaction Detection

Problem Statement:

Develop a Python program to process transaction IDs and amounts stored in a list, detect duplicate transactions, identify transactions that exceed a specified limit, and display suspicious transactions.

Objectives:

- Store transaction IDs and amounts.
- Detect duplicate transaction IDs.
- Identify transactions exceeding a specified amount.
- Display suspicious transactions.

Assumptions:

- The suspicious transaction limit is ₹10,000.
- A transaction ID appearing more than once is considered a duplicate.
- A transaction is suspicious if its amount exceeds ₹10,000.

Algorithm:

1. Start.
2. Input the number of transactions.
3. Enter each transaction ID and amount.
4. Check for duplicate transaction IDs.
5. Check whether each amount exceeds ₹10,000.
6. Display suspicious transactions.
7. Stop.

Flowchart:

Start → Input transaction details → Check duplicate IDs and amount limit → Identify suspicious transactions → Display results → Stop

Pseudocode:

```
BEGIN
  INPUT number_of_transactions

  FOR each transaction
```

```
    INPUT transaction_id and amount
    STORE transaction details
  END FOR

  FOR each transaction
    CHECK whether transaction ID is duplicated
    CHECK whether amount exceeds 10000

    IF duplicate OR amount > 10000
      DISPLAY suspicious transaction
    END IF
  END FOR
END
```

Python Source Code:

```
n = int(input("Enter number of transactions: "))

ids = []
amounts = []

for i in range(n):
  print("\nTransaction", i + 1)
  tid = input("Enter transaction ID: ")
  amount = float(input("Enter transaction amount: "))

  ids.append(tid)
  amounts.append(amount)
```

```
print("\nSuspicious Transactions:")

found = False

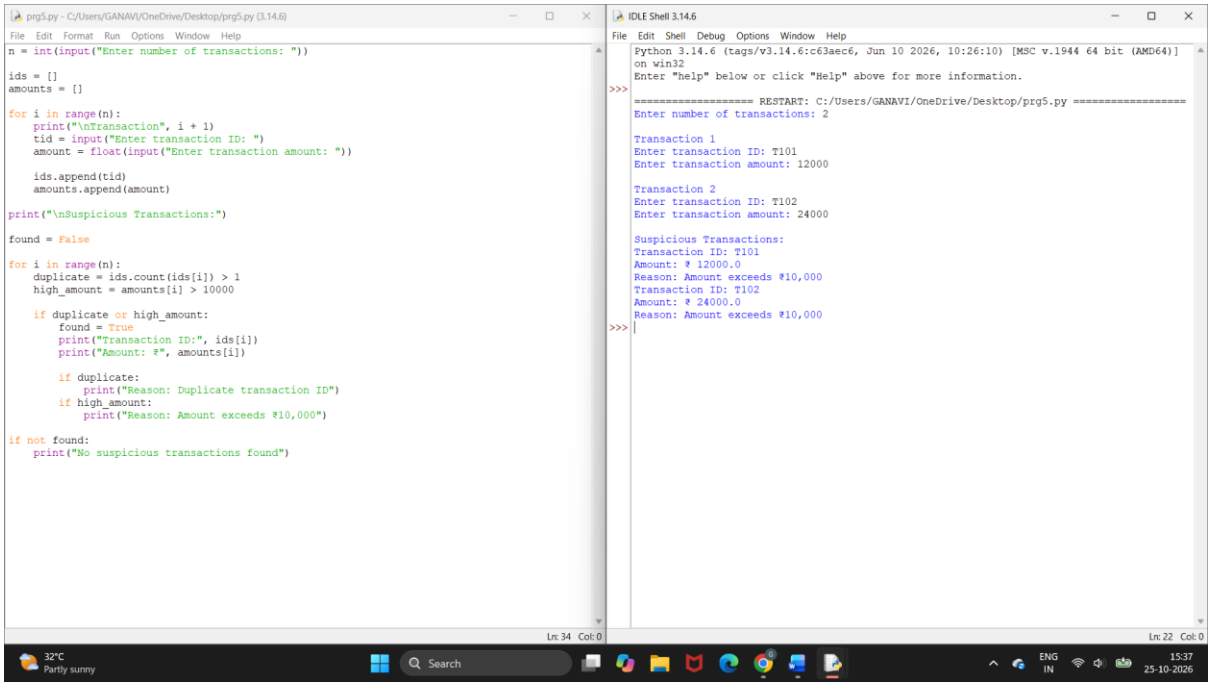
for i in range(n):
    duplicate = ids.count(ids[i]) > 1
    high_amount = amounts[i] > 10000

    if duplicate or high_amount:
        found = True
        print("Transaction ID:", ids[i])
        print("Amount: ₹", amounts[i])

        if duplicate:
            print("Reason: Duplicate transaction ID")
        if high_amount:
            print("Reason: Amount exceeds ₹10,000")

if not found:
    print("No suspicious transactions found")
```

output:



Testcases:

Testcases	Input	Expected output
1	Unique IDs, amounts below ₹10,000	No suspicious transactions
2	Duplicate transaction ID	Duplicate detected
3	Amount ₹15,000	High-value transaction detected

Conclusion:

This program detects duplicate transaction IDs and high-value transactions using lists, loops, conditional statements, and the count() function. It helps identify potentially suspicious transactions.

Program 6

Repeated Word and Phrase Detection

Problem Statement:

Process a paragraph, convert all words to lowercase, count the occurrences of each word using string and list operations, and identify repeated words along with their positions in the paragraph.

Objectives:

- Accept a paragraph from the user.
- Convert the paragraph to lowercase.
- Separate the paragraph into individual words.
- Count the occurrences of each word.
- Identify repeated words and display their positions.

Assumptions:

- Words are separated by spaces.
- Uppercase and lowercase versions of a word are treated as the same word.
- Basic punctuation is removed before counting.
- A word is repeated if it appears more than once.

Algorithm:

1. Start.
2. Input a paragraph.
3. Convert the paragraph to lowercase.
4. Remove basic punctuation and split the paragraph into words.
5. Count each word's occurrences.
6. Find repeated words and their positions.

7. Display the results.
8. Stop.

Flowchart:

Start → Input paragraph → Convert to lowercase → Remove punctuation and split into words → Count words and find positions → Display repeated words → Stop

Pseudocode:

```
BEGIN
  INPUT paragraph
  CONVERT paragraph to lowercase
  REMOVE punctuation from paragraph
  SPLIT paragraph into words

  FOR each word in words
    COUNT its occurrences
    IF count > 1
      FIND all positions of the word
      DISPLAY word, count, and positions
    END IF
  END FOR
END
```

Source code:

```
paragraph = input("Enter a paragraph: ")

paragraph = paragraph.lower()

punctuation = ".,!?;:"
```

```
for ch in punctuation:
    paragraph = paragraph.replace(ch, "")

words = paragraph.split()

print("\nRepeated Words:")

found = False

for word in words:
    if words.count(word) > 1 and word not in words[:words.index(word)]:
        positions = []

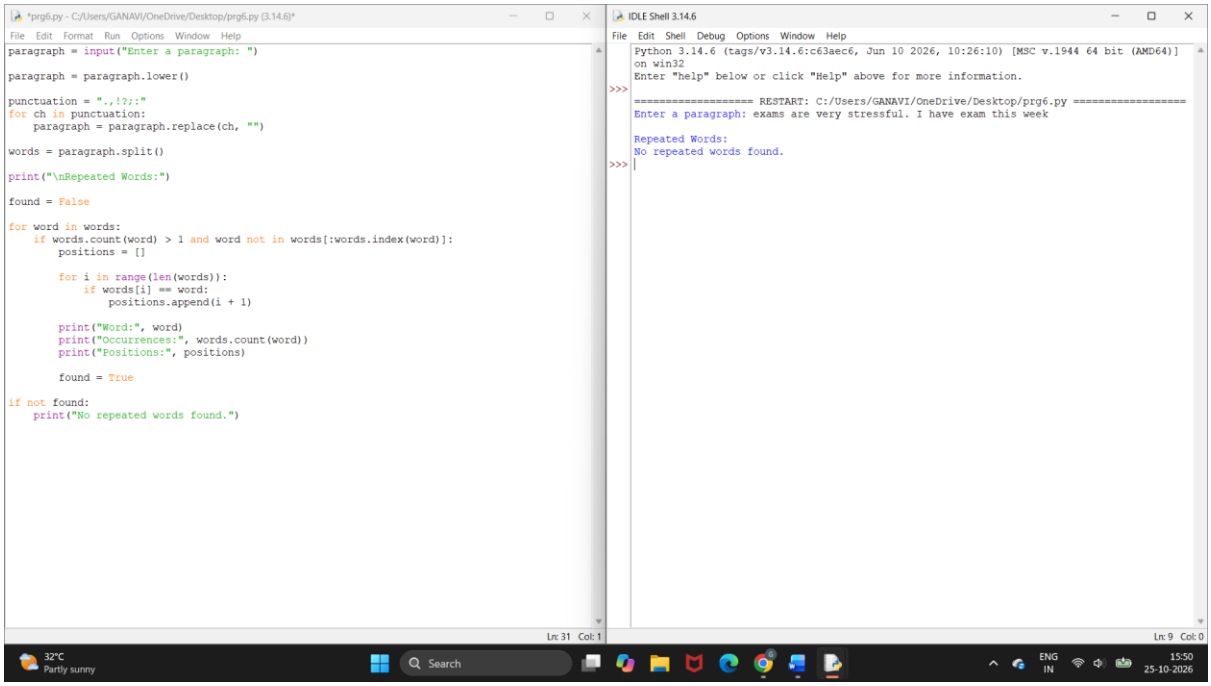
        for i in range(len(words)):
            if words[i] == word:
                positions.append(i + 1)

        print("Word:", word)
        print("Occurrences:", words.count(word))
        print("Positions:", positions)

        found = True

if not found:
    print("No repeated words found.")
```

output:



Testcases:

Testcases	Input	Expected output
1	Python is easy. Python is useful.	python and is are repeated
2	AI is useful	No repeated words
3	Hello hello HELLO	hello occurs 3 times

Conclusion:

This program detects repeated words in a paragraph by using Python strings, lists, loops, and conditional statements. It counts each repeated word and displays its positions, helping identify repetition in text.

Program7

Tuple-Based Route Distance Analysis

Problem Statement:

Store geographical coordinates as tuples, calculate distances between successive points using a supplied formula, and identify the longest route segment.

Objectives:

- Store geographical coordinates using tuples.
- Calculate distances between successive points.
- Identify the distance of each route segment.
- Find and display the longest route segment.

Assumptions:

- Each location is represented by a tuple containing its \(x\) and \(y\) coordinates.
- The distance is calculated using the Euclidean distance formula.
- At least two locations are entered.
- Coordinates are given in the same units.

Algorithm:

1. Start.
2. Input the number of locations.
3. Input the coordinates of each location and store them as tuples.
4. Calculate the distance between each pair of successive locations.
5. Compare the distances to find the longest segment.
6. Display the distances and the longest segment.
7. Stop.

Flowchart:

Start → Input location coordinates → Store coordinates as tuples → Calculate successive distances → Find the longest segment → Display results → Stop

Pseudocode:

```
BEGIN
  INPUT number of locations
  INPUT coordinates of each location as tuples
```

```
  SET longest_distance = 0

  FOR each successive pair of locations
    CALCULATE distance using the distance formula
    DISPLAY distance

    IF distance > longest_distance
      SET longest_distance = distance
      STORE the segment
    END IF
  END FOR

  DISPLAY longest segment and its distance
END
```

Source code:

```
import math

n = int(input("Enter number of locations: "))

locations = []

for i in range(n):
  x = float(input("Enter x coordinate: "))
  y = float(input("Enter y coordinate: "))
  locations.append((x, y))
```

```
longest_distance = 0
longest_segment = None

print("\nDistances between locations:")

for i in range(n - 1):
    x1, y1 = locations[i]
    x2, y2 = locations[i + 1]

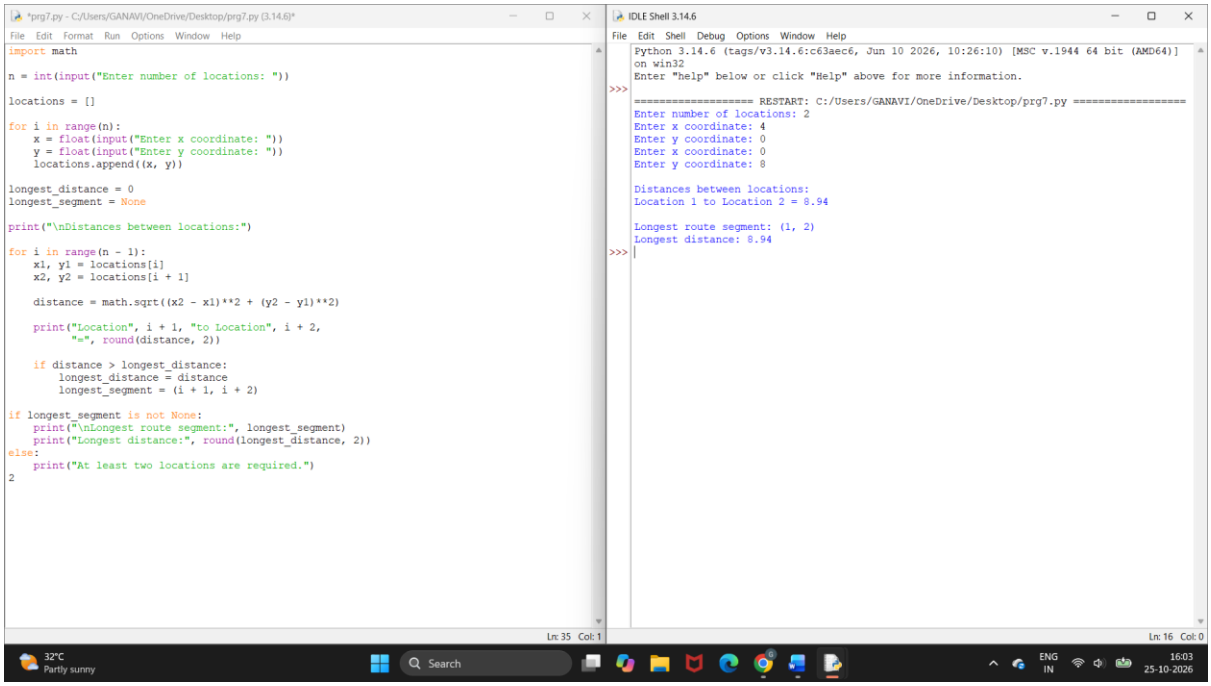
    distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

    print("Location", i + 1, "to Location", i + 2,
          "=", round(distance, 2))

    if distance > longest_distance:
        longest_distance = distance
        longest_segment = (i + 1, i + 2)

if longest_segment is not None:
    print("\nLongest route segment:", longest_segment)
    print("Longest distance:", round(longest_distance, 2))
else:
    print("At least two locations are required.")
```

output:



Testcases:

Testcases	Input	Expected output
1	(0,0), (3,4)	Distance = 5
2	(0,0), (3,4), (9,4)	Longest segment=6

Conclusion:

This program uses tuples to store geographical coordinates and the Euclidean distance formula to calculate distances between successive locations. It identifies the longest route segment using loops and conditional statements.

Program8

University Hostel Room Allocation

Problem Statement:

Allocate students to rooms according to room capacity, room eligibility, and specified grouping constraints. Detect duplicate assignments and calculate the number of students who remain unallocated.

Objectives:

- Allocate students to suitable hostel rooms.
- Ensure room capacity is not exceeded.
- Check student eligibility for each room.
- Detect duplicate student assignments.
- Identify students who could not be allocated a room.

Assumptions:

- Each student has a unique student ID.
- Each room has a fixed capacity.
- Students are assigned to rooms based on eligibility.
- A student can be assigned to only one room.
- Students who cannot be assigned remain on the unallocated list.

Algorithm:

1. Start.
2. Input student details and room information.
3. Store student records and room details in lists.
4. Check student eligibility for each room.
5. Assign eligible students while room capacity is available.
6. Detect duplicate assignments.
7. Display room-wise allocation and unallocated students.
8. Stop.

Flowchart:

Start → Input student and room details → Check eligibility and room capacity → Allocate students → Detect duplicate assignments → Display allocations and unallocated students → Stop

Pseudocode:

```
BEGIN
  INPUT student records
  INPUT room details

  INITIALIZE assigned_students as empty list
  INITIALIZE unallocated_students as empty list

  FOR each student
    CHECK whether student is already assigned

    IF student is already assigned
      DISPLAY duplicate assignment
    ELSE
      FIND an eligible room with available capacity

      IF a suitable room is found
        ASSIGN student to room
        UPDATE room occupancy
      ELSE
        ADD student to unallocated_students
      END IF
    END IF
  END FOR

  DISPLAY room-wise allocations
  DISPLAY unallocated_students
END
```

Source code:

```
students = [
    ("S101", "Asha", "Female"),
    ("S102", "Rahul", "Male"),
    ("S103", "Priya", "Female"),
    ("S104", "Arjun", "Male"),
    ("S105", "Sneha", "Female")
]

# Room: (room number, capacity, eligibility)
rooms = [
    ("R101", 2, "Female"),
    ("R102", 2, "Male")
]

allocations = {}
occupancy = {}

for room in rooms:
    occupancy[room[0]] = 0

assigned_ids = []
unallocated = []

for student in students:
    sid, name, category = student

    if sid in assigned_ids:
```

```
        print("Duplicate student assignment:", sid)
        continue

    assigned = False

    for room in rooms:
        room_no, capacity, eligibility = room

        if (category == eligibility and
                occupancy[room_no] < capacity):
            allocations[sid] = (name, room_no)
            occupancy[room_no] += 1
            assigned_ids.append(sid)
            assigned = True
            break

    if not assigned:
        unallocated.append((sid, name))

print("\nRoom-wise Allocation:")

for room in rooms:
    room_no = room[0]
    print("\nRoom:", room_no)

    found = False

    for sid, details in allocations.items():
```

```
        name, allocated_room = details

        if allocated_room == room_no:
            print(sid, "-", name)
            found = True

    if not found:
        print("No students assigned")

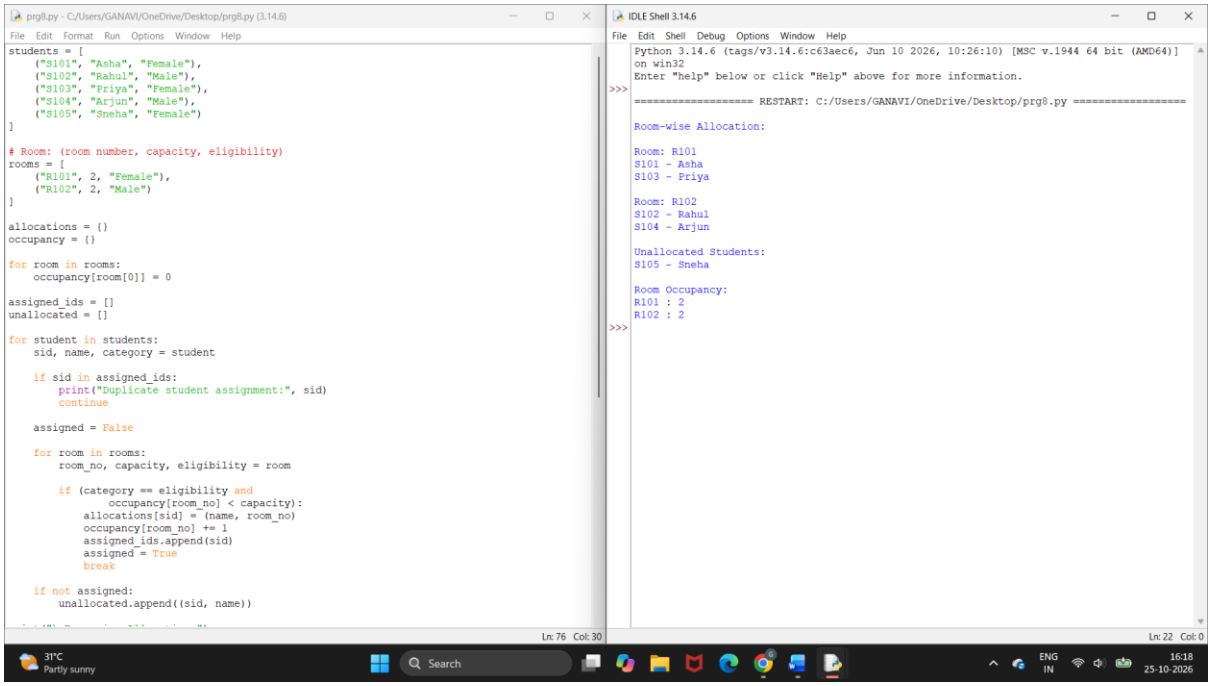
print("\nUnallocated Students:")

if unallocated:
    for student in unallocated:
        print(student[0], "-", student[1])
else:
    print("None")

print("\nRoom Occupancy:")

for room_no, count in occupancy.items():
    print(room_no, ":", count)
```

output:



Testcases:

Testcases	Input condition	Expected output
1	Eligible student and available room	Student allocated
2	Room is full	Student remains unallocated
3	Student category does not match room eligibility	Student remains unallocated
4	Student ID is already assigned	Student remains unallocated

Conclusion:

This program demonstrates university hostel room allocation using Python lists, tuples, loops, and conditional statements. It checks room capacity and eligibility, allocates students to suitable rooms, and identifies unallocated students.

Program9

Campus Resource Allocation Under Constraints

Problem statement:

Compare equipment requirements with available resources across multiple activities, calculate shortages, and produce a feasible allocation using a specified priority order.

Objectives:

- Identify the equipment required for each campus activity.
- Compare required quantities with available resources.
- Allocate resources according to activity priority.
- Calculate shortages and report unmet requirements.

Assumptions:

- Resources are shared among campus activities.
- Higher-priority activities receive resources first.
- An activity receives only the quantity available.
- Any unmet requirement is reported as a shortage.

Algorithm:

1. Start.
2. Enter the available quantity of each resource.
3. Enter activity names, priorities, and equipment requirements.
4. Arrange activities in priority order.
5. Check available resources for each activity.
6. Allocate the available quantity.
7. Calculate any shortage.
8. Display allocations and remaining resources.

9. Stop.

Flowchart:

Start → Enter resources and activity requirements → Arrange activities by priority → Check availability → Allocate resources and calculate shortages → Display results → Stop

Pseudocode:

```
START

INPUT available resources

INPUT activity requirements and priorities

SORT activities by priority


FOR each activity:
  FOR each required resource:
    IF available quantity >= required quantity:
      ALLOCATE required quantity
      UPDATE available quantity
    ELSE:
      ALLOCATE available quantity
      CALCULATE shortage
      SET available quantity to zero
  DISPLAY allocation and shortage


DISPLAY remaining resources

STOP
```

Python source code:

```
resources = {
  "Projector": 4,
```

```
    "Laptop": 6,
    "Microphone": 3
}

activities = [
    ("AI Workshop", 1, {"Projector": 2, "Laptop": 3}),
    ("Technical Seminar", 2, {"Projector": 2, "Microphone": 2}),
    ("Coding Competition", 3, {"Laptop": 5, "Microphone": 2})
]

activities.sort(key=lambda x: x[1])

print("CAMPUS RESOURCE ALLOCATION")
print("-" * 35)

for name, priority, requirements in activities:
    print("\nActivity:", name)
    print("Priority:", priority)

    for resource, required in requirements.items():
        available = resources.get(resource, 0)
        allocated = min(required, available)
        shortage = required - allocated

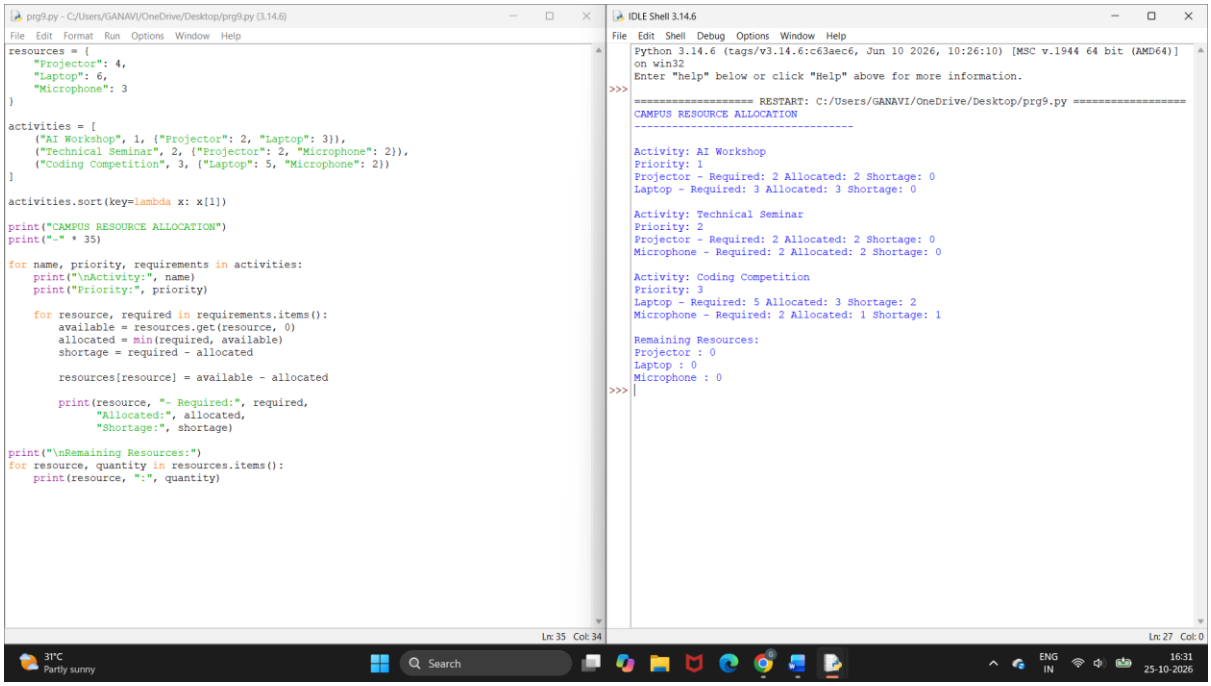
        resources[resource] = available - allocated

        print(resource, "- Required:", required,
              "Allocated:", allocated,
```

```
        "Shortage:", shortage)

print("\nRemaining Resources:")
for resource, quantity in resources.items():
    print(resource, ":", quantity)
```

output:



Testcases:

testcases	Condition	Excepted output
1	Available resources meet all requirements	No shortage
2	Available resources are insufficient	Shortage is displayed
3	Two activities require the same resource	Higher-priority activity receives resources first

Conclusion:

The Campus Resource Allocation system efficiently distributes limited equipment among multiple campus activities according to priority. It calculates shortages, updates remaining resources, and helps ensure that important activities receive resources first.

Program10

Railway Fare and Reservation Calculator

Problem statement:

Calculate railway fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties. The program should calculate the total fare for each passenger and determine the cancellation penalty when applicable.

Objectives:

- Calculate fares based on journey distance and travel class.
- Apply age-based concessions.
- Add reservation fees.
- Calculate cancellation penalties.
- Display the final fare for each passenger.

Assumptions:

- The basic fare per kilometre depends on the travel class.
- Children below 5 years travel free in this simplified example.
- Passengers aged 60 years or above receive a 10% concession.
- A fixed reservation fee is added to the fare.
- The cancellation penalty is 20% of the basic fare.
- These are sample rules for the assignment, not actual Indian Railways fare rules.

Algorithm:

1. Start.
2. Input the passenger's name, age, distance, and travel class.
3. Select the fare rate based on the travel class.
4. Calculate the basic fare using distance and fare rate.
5. Apply the age-based concession, if eligible.
6. Add the reservation fee.
7. Calculate the cancellation penalty.
8. Display the fare details.
9. Stop.

Flowchart:

```
Start → Input passenger details → Select travel class and fare rate →
Calculate basic fare → Apply age concession → Add reservation fee →
Calculate cancellation penalty → Display fare details → Stop
```

Pseudocode:

```
START
INPUT number of passengers

FOR each passenger:
    INPUT name, age, distance, travel class

    IF distance <= 100:
        CALCULATE fare using first distance slab
    ELSE IF distance <= 500:
        CALCULATE fare using second distance slab
    ELSE:
        CALCULATE fare using third distance slab

    APPLY travel class multiplier

    IF age < 5:
        BASIC FARE = 0
    ELSE IF age >= 60:
        APPLY 10% concession

    ADD reservation fee
    CALCULATE cancellation penalty
    DISPLAY passenger fare details

STOP
```

Python source code:

```
print("RAILWAY FARE AND RESERVATION CALCULATOR")



n = int(input("Enter number of passengers: "))
```

```
for i in range(n):
    print("\nPassenger", i + 1)

    name = input("Enter passenger name: ")
    age = int(input("Enter age: "))
    distance = float(input("Enter distance in km: "))

    print("Classes: 1. Sleeper  2. AC  3. First Class")
    choice = int(input("Select class (1-3): "))

    if choice == 1:
        class_name = "Sleeper"
        multiplier = 1.0
        reservation = 40
    elif choice == 2:
        class_name = "AC"
        multiplier = 1.5
        reservation = 80
    elif choice == 3:
        class_name = "First Class"
        multiplier = 2.0
        reservation = 100
    else:
        print("Invalid class selection.")
        continue

    # Distance-based basic fare
```

```
if distance <= 100:
   base_fare = distance * 2
elif distance <= 500:
   base_fare = 100 * 2 + (distance - 100) * 1.5
else:
   base_fare = (100 * 2 + 400 * 1.5
            + (distance - 500) * 1.2)

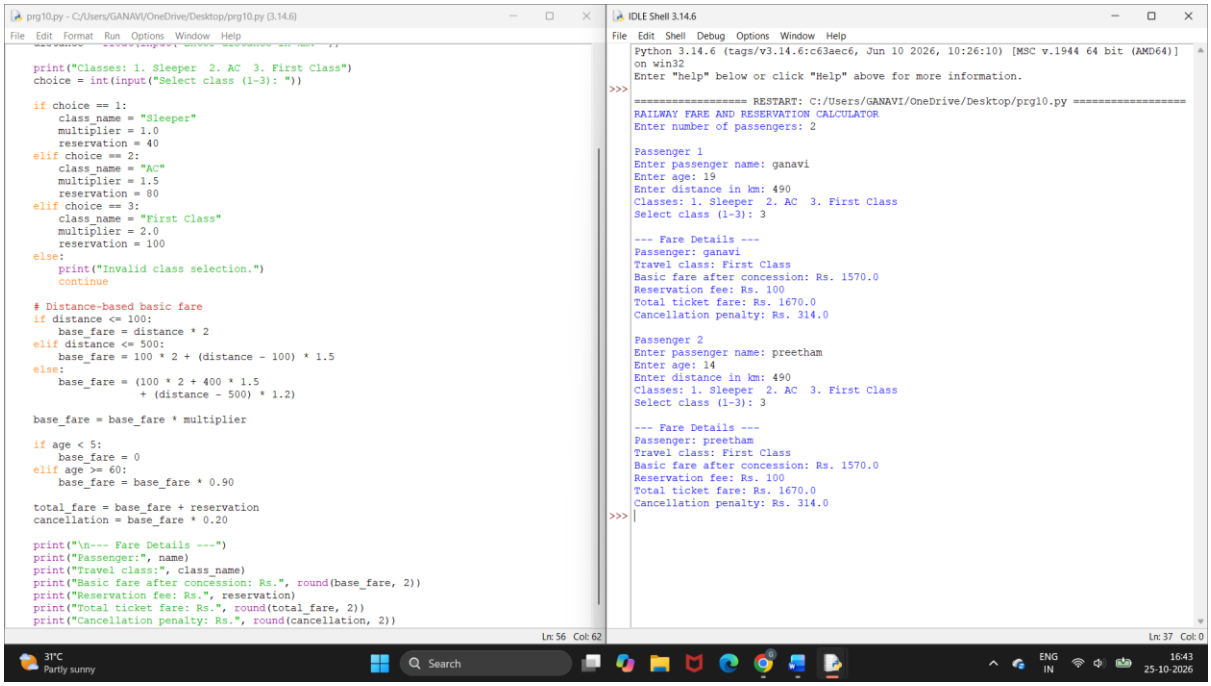
base_fare = base_fare * multiplier

if age < 5:
   base_fare = 0
elif age >= 60:
   base_fare = base_fare * 0.90

total_fare = base_fare + reservation
cancellation = base_fare * 0.20

print("\n--- Fare Details ---")
print("Passenger:", name)
print("Travel class:", class_name)
print("Basic fare after concession: Rs.", round(base_fare, 2))
print("Reservation fee: Rs.", reservation)
print("Total ticket fare: Rs.", round(total_fare, 2))
print("Cancellation penalty: Rs.", round(cancellation, 2))
```

output:



Testcases:

Testcases	Input condition	Expected output
1	Distance = 80 km, Sleeper	Fare calculated using first slab
2	Distance = 200 km, AC	Fare calculated using second slab and AC multiplier
3	Passenger age = 4	Basic fare is zero; reservation fee is added
4	Passenger age = 65	10% concession applied

Conclusion:

The Railway Fare and Reservation Calculator calculates passenger fares using distance slabs, travel classes, age-based concessions, and reservation fees. It also estimates cancellation penalties, making the fare calculation process simple and organized.